

建立僅支援 IPv6 的 VM 執行個體，並啟用 NAT64/DNS64

來源：<https://codelabs.developers.google.com/ipv6-publicnat64-dns64?hl=zh-tw>

步驟數：9

在本程式碼研究室中，您將測試 Google Cloud 中僅限 IPv6 的 VM、NAT64 和 DNS64 服務功能，重點是僅限 IPv6 的子網路和執行個體。您將探索僅限 IPv6 的執行個體如何使用這些服務存取 IPv4 資源，並透過 DNS64 檢查 DNS 解析行為。

目錄

1. [1. 簡介](#)
2. [2. 學習內容](#)
3. [3. 事前準備](#)
4. [4. 準備步驟](#)
5. [5. 檢查僅支援 IPv6 的執行個體。](#)
6. [6. 啟用 NAT64 和 DNS64](#)
7. [7. 測試 NAT64 和 DNS64](#)
8. [8. 清理](#)
9. [9. 恭喜](#)

步驟 1 · 約 0 分鐘

建立僅支援 IPv6 的 VM 執行個體，並啟用 NAT64/DNS64

程式碼研究室簡介

*subject*上次更新時間：3月 27, 2026

*account_circle*作者：Ghaleb Al-Habian

1. 簡介

遷移至 IPv6 的主要挑戰之一，是維持僅支援 IPv4 的端點和網路的可連線性。為解決這項挑戰，目前最先進的技術是結合使用 DNS64 (定義於 RFC6147)，將用戶端的 A 記錄轉譯為 AAAA 記錄，然後結合 NAT64 (定義於 RFC6146)，將特殊格式的 IPv6 位址轉譯為 IPv4，其中 IPv4 位址會嵌入特殊 IPv6 位址。本程式碼實驗室會引導使用者在 Google Cloud Platform (GCP) 虛擬私有雲 (VPC) 中設定這兩項功能。一併設定 GCP NAT64 和 DNS64 後，僅支援 IPv6 的執行個體就能與網際網路上僅支援 IPv4 的伺服器通訊。

在本實驗室中，您將設定虛擬私有雲，並使用不同類型的 IPv6 子網路和執行個體：僅支援 IPv6 的 GUA (全域單點播送位址)、僅支援 IPv6 的 ULA (專屬本機位址) 和雙重堆疊 ULA。接著，您將設定及測試 Google Cloud 的受管理 DNS64 和 NAT64 服務，以便從這些服務存取僅限 IPv4 的網站。

2. 學習內容

- 如何建立僅支援 IPv6 的子網路和執行個體
 - 如何為虛擬私有雲啟用 Google Cloud 的代管 DNS64 服務。
 - 如何建立已設定 NAT64 的 Google Cloud NAT 閘道。
 - 如何從僅支援 IPv6 的執行個體測試 DNS64 解析，連線至僅支援 IPv4 的目的地。
 - 單一堆疊和雙重堆疊執行個體之間的 DNS64 和 NAT64 行為差異。
 - 如何為 NAT64 設定 NAT 閘道。
 - 如何從僅支援 IPv6 的執行個體測試 NAT64 連線至僅支援 IPv4 的目的地。
-

步驟 3 · 約 0 分鐘

3. 事前準備

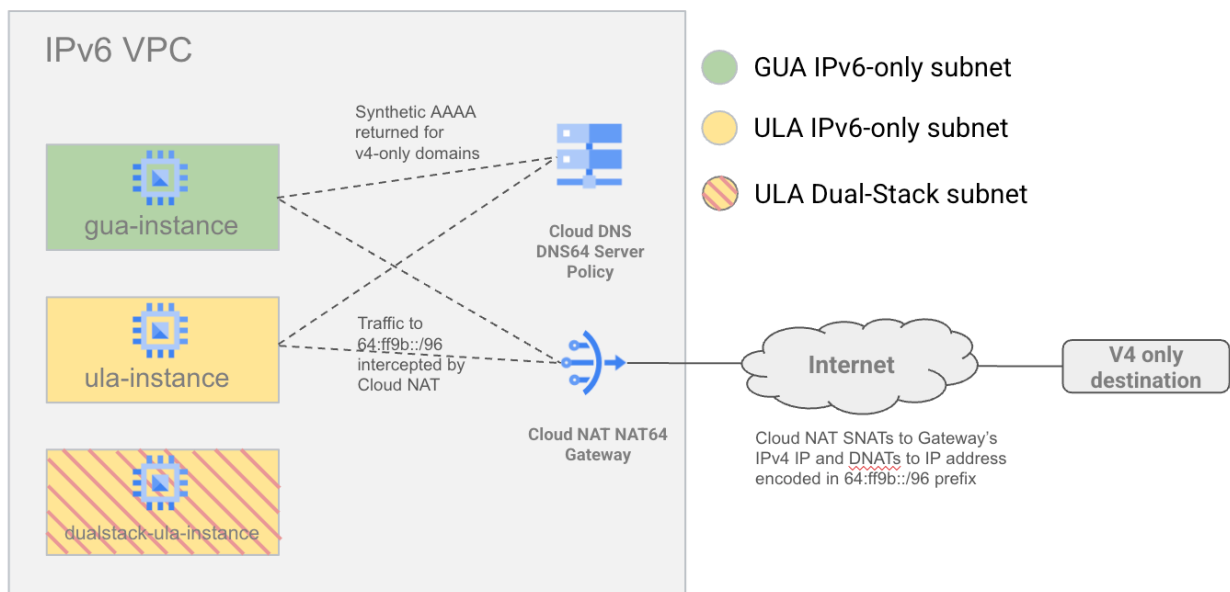
更新專案以支援 Codelab

本程式碼研究室會使用 `$variables`，協助您在 Cloud Shell 中實作 `gcloud` 設定。

在 Cloud Shell 中執行下列操作：

```
gcloud config list project
gcloud config set project [YOUR-PROJECT-ID]
export projectname=$(gcloud config list --format="value(core.project)")
export zonename=[COMPUTE_ZONE_NAME]
export regionname=[REGION_NAME]
```

實驗室整體架構



為示範 NAT64 和 DNS64 如何與不同 IPv6 子網路類型互動，您將建立單一虛擬私有雲，其中包含 GUA 和 ULA 類型的 IPv6 子網路。您也會建立雙重堆疊子網路 (使用 ULA 位址)，示範 DNS64 和 NAT64 如何不適用於雙重堆疊 VM。

接著，您將設定 DNS64 和 NAT64，並測試與網際網路上 IPv6 和 IPv4 目的地的連線。

4. 準備步驟

首先，請在 Google Cloud 專案中設定必要的服務帳戶、IAM、網路基礎架構和執行個體。

建立服務帳戶和 IAM 繫結

首先，我們要建立新的服務帳戶，讓執行個體能使用 gcloud 透過 SSH 互相連線。我們需要這項功能，因為無法使用 IAP 存取 GUA 僅支援 IPv6 的執行個體，且 Cloud Shell 尚不允許直接存取 IPv6。在 Cloud Shell 中執行下列指令。

〇〇〇〇

```
gcloud iam service-accounts create ipv6-codelab \
  --description="temporary service account for a codelab" \
  --display-name="ipv6codelabSA" \
  --project $projectname

gcloud projects add-iam-policy-binding $projectname \
  --member=serviceAccount:ipv6-codelab@$projectname.iam.gserviceaccount.com \
  --role=roles/compute.instanceAdmin.v1

gcloud iam service-accounts add-iam-policy-binding \
  ipv6-codelab@$projectname.iam.gserviceaccount.com \
  --member=serviceAccount:ipv6-codelab@$projectname.iam.gserviceaccount.com \
  --role=roles/iam.serviceAccountUser
```

建立虛擬私有雲並啟用 ULA

在 Cloud Shell 中執行下列指令，建立採用自訂子網路模式的虛擬私有雲網路，並啟用 ULA 內部 IPv6。

〇〇〇〇

```
gcloud compute networks create ipv6-only-vpc \
  --project=$projectname \
  --subnet-mode=custom \
  --mtu=1500 --bgp-routing-mode=global \
  --enable-ula-internal-ipv6
```

建立防火牆規則

建立防火牆規則，允許 SSH 存取。其中一項規則允許來自整體 ULA 範圍 (fd20::/20) 的 SSH 連線。另外兩條規則允許來自 IAP 預先定義的 IPv6 和 IPv4 範圍 (分別為 2600:2d00:1:7::/64 和 35.235.240.0/20) 的流量。

在 Cloud Shell 中執行下列指令：

```
gcloud compute firewall-rules create allow-v6-ssh-ula \
--direction=INGRESS --priority=200 \
--network=ipv6-only-vpc --action=ALLOW \
--rules=tcp:22 --source-ranges=fd20::/20 \
--project=$projectname
```

```
gcloud compute firewall-rules create allow-v6-iap \
--direction=INGRESS --priority=300 \
--network=ipv6-only-vpc --action=ALLOW \
--rules=tcp --source-ranges=2600:2d00:1:7::/64 \
--project=$projectname
```

```
gcloud compute firewall-rules create allow-v4-iap \
--direction=INGRESS --priority=300 \
--network=ipv6-only-vpc --action=ALLOW \
--rules=tcp --source-ranges=35.235.240.0/20 \
--project=$projectname
```

建立子網路

建立僅限 GUA v6 的子網路、僅限 ULA v6 的子網路，以及雙重堆疊 ULA 子網路。在 Cloud Shell 中執行下列指令：

```
gcloud compute networks subnets create gua-v6only-subnet \
--network=ipv6-only-vpc \
--project=$projectname \
--stack-type=IPV6_ONLY \
--ipv6-access-type=external \
--region=$regionname
```

```
gcloud compute networks subnets create ula-v6only-subnet \
--network=ipv6-only-vpc \
--project=$projectname \
--stack-type=IPV6_ONLY \
--ipv6-access-type=internal \
--enable-private-ip-google-access \
--region=$regionname
```

```
gcloud compute networks subnets create ula-dualstack-subnet \
--network=ipv6-only-vpc \
--project=$projectname \
--stack-type=IPV4_IPV6 \
--range=10.120.0.0/16 \
--ipv6-access-type=internal \
--region=$regionname
```

建立執行個體

在您剛建立的每個子網路中建立執行個體。指定僅支援 IPv6 的 ULA 執行個體時，請使用 cloud-platform，這樣我們就能將該執行個體當做跳板，連線至僅支援 IPv6 的 GUA 執行個體。在 Cloud Shell 中執行下列指令：

```
gcloud compute instances create gua-instance \
--subnet gua-v6only-subnet \
--stack-type IPV6_ONLY \
--zone $zonename \
--scopes=https://www.googleapis.com/auth/cloud-platform \
--service-account=ipv6-codelab@$projectname.iam.gserviceaccount.com \
--project=$projectname

gcloud compute instances create ula-instance \
--subnet ula-v6only-subnet \
--stack-type IPV6_ONLY \
--zone $zonename \
--scopes=https://www.googleapis.com/auth/cloud-platform \
--service-account=ipv6-codelab@$projectname.iam.gserviceaccount.com \
--project=$projectname

gcloud compute instances create dualstack-ula-instance \
--subnet ula-dualstack-subnet \
--stack-type IPV4_IPV6 \
--zone $zonename \
--project=$projectname
```

初始執行個體存取權和設定

透過 SSH 連線至 ULA 執行個體，系統預設會使用 IAP。在 Cloud Shell 中使用下列指令，透過 SSH 連線至 ULA 執行個體：

```
gcloud compute ssh ula-instance --project $projectname --zone $zonename

<username>@ula-instance:~$
```

我們也會將 ULA 執行個體做為 GUA 執行個體的跳板 (因為 IAP 無法搭配 GUA 執行個體使用，且 Cloud Shell VM 無法存取 IPv6 目的地)。

仍位於 ULA 執行個體殼層中。使用下列 gcloud 指令，嘗試透過 SSH 連線至 GUA 執行個體。

首次在執行個體內執行 SSH 指令時，系統會提示您設定 SSH 金鑰組。持續按 Enter 鍵，直到金鑰建立完成並執行 SSH 指令為止；這會建立沒有通關密語的新金鑰組。


```
ula-instance:~$ gcloud compute ssh gua-instance
```

WARNING: The private SSH key file for gcloud does not exist.

WARNING: The public SSH key file for gcloud does not exist.

WARNING: You do not have an SSH key for gcloud.

WARNING: SSH keygen will be executed to generate a key.

Generating public/private rsa key pair.

Enter passphrase (empty for no passphrase):

Enter same passphrase again:

Your identification has been saved in /home/galhabian/.ssh/google_compute_engine

Your public key has been saved in /home/galhabian/.ssh/google_compute_engine.pub

The key fingerprint is:

SHA256:5PYzYdjcpWYiFtZetYCBi6vmy9dqyLsxgDORkB9ynqY galhabian@ula-instance

The key's randomart image is:

```
+---[RSA 3072]-----+
|..                |
|+.O      .        |
|o= o  . + .        |
| o=    * o o        |
|+o.    . S o . o    |
|Eo . . . O + = .    |
|  .=. .+ @ * .      |
|  +OO... *          |
|   **..             |
+-----[SHA256]-----+
```

如果成功，SSH 指令就會成功，您也會成功透過 SSH 連線至 GUA 執行個體：

Updating instance ssh metadata...done.

Waiting for SSH key to propagate.

Warning: Permanently added 'compute.3639038240056074485' (ED25519) to the list of known hosts.

Linux gua-instance 6.1.0-34-cloud-amd64 #1 SMP PREEMPT_DYNAMIC Debian 6.1.135-1 (2025-04-25)
x86_64

The programs included with the Debian GNU/Linux system are free software;
the exact distribution terms for each program are described in the
individual files in /usr/share/doc/*/copyright.

Debian GNU/Linux comes with ABSOLUTELY NO WARRANTY, to the extent
permitted by applicable law.

<username>@gua-instance:~\$

附註：請先結束所有 SSH 工作階段，再繼續操作。

步驟 5 · 約 0 分鐘

5. 檢查僅支援 IPv6 的執行個體。

我們將透過 SSH 連線至這兩個僅支援 IPv6 的執行個體，並檢查其路由表。

檢查 GUA 執行個體

先跳過「ula-instance」執行個體，再透過 SSH 連線至「gua-instance」。

```
gcloud compute ssh ula-instance --project $projectname --zone $zonename  
<username>@ula-instance:~$ gcloud compute ssh gua-instance
```

讓我們使用下列指令查看執行個體的路由表

```
<username>@gua-instance:~$ ip -6 route  
  
2600:1900:4041:461::/65 via fe80::56:11ff:fef9:88c1 dev ens4 proto ra metric 100 expires  
81sec pref medium  
fe80::/64 dev ens4 proto kernel metric 256 pref medium  
default via fe80::56:11ff:fef9:88c1 dev ens4 proto ra metric 100 expires 81sec mtu 1500 pref  
medium
```

我們注意到路由表中有三個項目

1. 執行個體所屬 GUA 子網路的 /65 路由，以及使用預設閘道連結本機位址的衍生下一個躍點。請注意，/65 以上的範圍保留給 IPv6 直通式網路負載平衡器
2. 連結本機單點播送前置字串 fe80::/64 的內建 /64 路由
3. 指向子網路預設閘道連結本機位址的預設路徑。

執行下列指令，查看 IPv4 路由表

```
<username>@gua-instance:~$ ip -4 route  
  
default via 169.254.1.1 dev ens4 proto dhcp src 169.254.1.2 metric 100  
169.254.1.1 dev ens4 proto dhcp scope link src 169.254.1.2 metric 100  
169.254.169.254 via 169.254.1.1 dev ens4 proto dhcp src 169.254.1.2 metric 100
```

出乎意料嗎？事實上，我們會在僅限 IPv6 的執行個體中維護 IPv4 路由表，嚴格來說是為了允許存取 [Compute 中繼資料伺服器](#) (169.254.169.154)，因為這仍是僅限 IPv4 的端點。

因為執行個體是僅支援 IPv6 的執行個體，因此會採用 IP 169.254.1.2。這個 IP 只能路由至 Compute 中繼資料伺服器，因此執行個體實際上會與所有 IPv4 網路隔離。

Curl 測試

我們將使用 curl 測試與僅限 v4 和僅限 v6 的網站的實際連線。

```
<username>@gua-instance:~$ curl -vv --connect-timeout 10 v6.ipv6test.app  
<username>@gua-instance:~$ curl -vv --connect-timeout 10 v4.ipv6test.app
```

以下是輸出範例。

```
<username>@gua-instance:~$ curl -vv --connect-timeout 10 v6.ipv6test.app
*   Trying [2600:9000:20be:cc00:9:ec55:a1c0:93a1]:80...
* Connected to v6.ipv6test.app (2600:9000:20be:cc00:9:ec55:a1c0:93a1) port 80 (#0)
> GET / HTTP/1.1
> Host: v6.ipv6test.app
> User-Agent: curl/7.88.1
> Accept: */*
>
< HTTP/1.1 200 OK
!! Rest of output truncated

<username>@gua-instance:~$ curl -vv --connect-timeout 10 v4.ipv6test.app
*   Trying 3.163.165.4:80...
* ipv4 connect timeout after 4985ms, move on!
*   Trying 3.163.165.50:80...
* ipv4 connect timeout after 2492ms, move on!
*   Trying 3.163.165.127:80...
* ipv4 connect timeout after 1246ms, move on!
*   Trying 3.163.165.37:80...
* ipv4 connect timeout after 1245ms, move on!
* Failed to connect to v4.ipv6test.app port 80 after 10000 ms: Timeout was reached
* Closing connection 0
curl: (28) Failed to connect to v4.ipv6test.app port 80 after 10000 ms: Timeout was reached
```

如預期，僅支援 IPv6 的執行個體無法連線至 IPv4 網際網路端點。如果未佈建 DNS64 和 NAT64，僅支援 IPv6 的執行個體就無法連線至 IPv4 目的地。由於執行個體具有 GUA IPv6 位址，因此可正常連線至 IPv6 目的地。

附註：請先結束所有 SSH 工作階段，再繼續操作。

檢查 ULA 執行個體

透過 SSH 連入「ula-instance」執行個體 (預設使用 IAP)。

```
gcloud compute ssh ula-instance --project $projectname --zone $zonename
```

讓我們使用下列指令查看執行個體的 IPv6 路由表

```
<username>@ula-instance:~$ ip -6 route

fd20:f06:2e5e:2000::/64 via fe80::55:82ff:fe6b:1d7 dev ens4 proto ra metric 100 expires 84sec
pref medium
fe80::/64 dev ens4 proto kernel metric 256 pref medium
default via fe80::55:82ff:fe6b:1d7 dev ens4 proto ra metric 100 expires 84sec mtu 1500 pref
medium
```

我們注意到路由表中有三個項目，與 GUA 執行個體類似，但遮罩為 /64 而非 /65。子網路路由屬於 ULA 範圍。(在 fd20::/20 匯總下)

執行下列指令，查看 IPv4 路由表

```
<username>@ula-instance:~$ ip -4 route

default via 169.254.1.1 dev ens4 proto dhcp src 169.254.1.2 metric 100
169.254.1.1 dev ens4 proto dhcp scope link src 169.254.1.2 metric 100
169.254.169.254 via 169.254.1.1 dev ens4 proto dhcp src 169.254.1.2 metric 100
```

這與 GUA 執行個體的情況類似。

Curl 測試

使用 curl 重複執行連線測試，測試僅支援 v4 和僅支援 v6 的網站。

```
<username>@ula-instance:~$ curl -vv --connect-timeout 10 v6.ipv6test.app
<username>@ula-instance:~$ curl -vv --connect-timeout 10 v4.ipv6test.app
```

以下是輸出範例。

```
<username>@ula-instance:~$ curl -vv --connect-timeout 10 v6.ipv6test.app
* Trying [2600:9000:20be:8400:9:ec55:alc0:93a1]:80...
* ipv6 connect timeout after 4986ms, move on!
* Trying [2600:9000:20be:9000:9:ec55:alc0:93a1]:80...
* ipv6 connect timeout after 2493ms, move on!
* Trying [2600:9000:20be:d600:9:ec55:alc0:93a1]:80...
* ipv6 connect timeout after 1246ms, move on!
* Trying [2600:9000:20be:b000:9:ec55:alc0:93a1]:80...
* ipv6 connect timeout after 622ms, move on!
* Trying [2600:9000:20be:7200:9:ec55:alc0:93a1]:80...
* ipv6 connect timeout after 312ms, move on!
* Trying [2600:9000:20be:8600:9:ec55:alc0:93a1]:80...
* ipv6 connect timeout after 155ms, move on!
* Trying [2600:9000:20be:7a00:9:ec55:alc0:93a1]:80...
* ipv6 connect timeout after 77ms, move on!
* Trying [2600:9000:20be:ce00:9:ec55:alc0:93a1]:80...
* ipv6 connect timeout after 77ms, move on!
* Failed to connect to v6.ipv6test.app port 80 after 10000 ms: Timeout was reached
* Closing connection 0

<username>@ula-instance:~$ curl -vv --connect-timeout 10 v4.ipv6test.app
* Trying 3.163.165.4:80...
* ipv4 connect timeout after 4985ms, move on!
* Trying 3.163.165.50:80...
* ipv4 connect timeout after 2492ms, move on!
* Trying 3.163.165.127:80...
* ipv4 connect timeout after 1246ms, move on!
* Trying 3.163.165.37:80...
* ipv4 connect timeout after 1245ms, move on!
* Failed to connect to v4.ipv6test.app port 80 after 10000 ms: Timeout was reached
* Closing connection 0
curl: (28) Failed to connect to v4.ipv6test.app port 80 after 10000 ms: Timeout was reached
```

在 ULA 執行個體案例中，由於 IPv6 端點無法使用 ULA 位址對外通訊，且執行個體僅支援 IPv6，因此無法連線至網際網路端點。

附註：請先結束所有 SSH 工作階段，再繼續操作。

6. 啟用 NAT64 和 DNS64

為 VPC 設定代管型 DNS64 和 NAT64 服務。

DNS64

為虛擬私有雲啟用 DNS64 伺服器政策。這會指示虛擬私有雲的 DNS 解析器，針對僅限 A 的回應合成 AAAA 記錄。在 Cloud Shell 中執行下列指令：

```
gcloud beta dns policies create allow-dns64 \
  --description="Enable DNS64 Policy" \
  --networks=ipv6-only-vpc \
  --enable-dns64-all-queries \
  --project $projectname
```

NAT64

建立 Cloud Router，這是 Cloud NAT 的必要條件。接著，建立設定為 NAT64 的 Cloud NAT 閘道，為所有僅支援 IPv6 的子網路 IP 範圍啟用，並自動分配外部 IP。在 Cloud Shell 中執行下列指令：

```
gcloud compute routers create nat64-router \
  --network=ipv6-only-vpc \
  --region=$regionname \
  --project=$projectname

gcloud beta compute routers nats create nat64-natgw \
  --router=nat64-router \
  --region=$regionname \
  --auto-allocate-nat-external-ips \
  --nat64-all-v6-subnet-ip-ranges \
  --project=$projectname
```

7. 測試 NAT64 和 DNS64

現在，讓我們從僅限 IPv6 的執行個體測試 NAT64 和 DNS64 設定，先從 GUA 執行個體開始，接著是 ULA 執行個體。

從 GUA 執行個體測試 DNS64/NAT64

先跳過「ula-instance」執行個體，再透過 SSH 連線至「gua-instance」。

```
gcloud compute ssh ula-instance --project $projectname --zone $zonename  
<username>@ula-instance:~$ gcloud compute ssh gua-instance
```

DNS 測試

測試僅支援 IPv6 的網站 DNS 解析情形 (例如 [v6.ipv6test.app](#)，但任何僅支援 IPv6 的網站都應產生類似結果)。

```
<username>@gua-instance:~$ host -t AAAA v6.ipv6test.app  
<username>@gua-instance:~$ host -t A v6.ipv6test.app
```

預期只會傳回 IPv6 AAAA 回覆。

輸出範例

```
<username>@gua-instance:~$ host -t AAAA v6.ipv6test.app  
v6.ipv6test.app has IPv6 address 2600:9000:269f:1000:9:ec55:a1c0:93a1  
v6.ipv6test.app has IPv6 address 2600:9000:269f:6600:9:ec55:a1c0:93a1  
v6.ipv6test.app has IPv6 address 2600:9000:269f:b600:9:ec55:a1c0:93a1  
v6.ipv6test.app has IPv6 address 2600:9000:269f:3e00:9:ec55:a1c0:93a1  
v6.ipv6test.app has IPv6 address 2600:9000:269f:9c00:9:ec55:a1c0:93a1  
v6.ipv6test.app has IPv6 address 2600:9000:269f:b200:9:ec55:a1c0:93a1  
v6.ipv6test.app has IPv6 address 2600:9000:269f:a600:9:ec55:a1c0:93a1  
v6.ipv6test.app has IPv6 address 2600:9000:269f:1400:9:ec55:a1c0:93a1  
  
<username>@gua-instance:~$ host -t A v6.ipv6test.app  
v6.ipv6test.app has no A record
```

測試僅支援 IPv4 的網站 (例如 [v4.ipv6test.app](#)) 的 DNS 解析。您預期會收到 A 記錄 (原始 IPv4) 和 DNS64 使用已知前置碼 64:ff9b::/96 合成的 AAAA 記錄。

```
<username>@gua-instance:~$ host -t AAAA v4.ipv6test.app  
<username>@gua-instance:~$ host -t A v4.ipv6test.app
```

輸出範例


```
<username>@gua-instance:~$ host -t AAAA v4.ipv6test.app
v4.ipv6test.app has IPv6 address 64:ff9b::36c0:3318
v4.ipv6test.app has IPv6 address 64:ff9b::36c0:3344
v4.ipv6test.app has IPv6 address 64:ff9b::36c0:333c
v4.ipv6test.app has IPv6 address 64:ff9b::36c0:3326

<username>@gua-instance:~$ host -t A v4.ipv6test.app
v4.ipv6test.app has address 54.192.51.68
v4.ipv6test.app has address 54.192.51.24
v4.ipv6test.app has address 54.192.51.60
v4.ipv6test.app has address 54.192.51.38
```

在上述範例中，十進位的 IPv4 位址 (54.192.51.38) 會轉換為十六進位的 (36 c0 33 26)，因此我們預期 AAAA 記錄的答案會是 (64:ff9b::36c0:3326)，這與我們收到的其中一個 AAAA 答案相符。

Curl 測試

讓我們使用透過 IPv6 的 curl，測試與相同 v4 專用和 v6 專用端點的實際連線

```
<username>@gua-instance:~$ curl -vv -6 v6.ipv6test.app

<username>@gua-instance:~$ curl -vv -6 v4.ipv6test.app
```

以下是輸出範例。

```
<username>@gua-instance:~$ curl -vv -6 v6.ipv6test.app
* Trying [2600:9000:269f:1000:9:ec55:a1c0:93a1]:80...
* Connected to v6.ipv6test.app (2600:9000:269f:1000:9:ec55:a1c0:93a1) port 80 (#0)
> GET / HTTP/1.1

##
## <Output truncated for brevity>
##

<username>@gua-instance:~$ curl -vv -6 v4.ipv6test.app
* Trying [64:ff9b::36c0:333c]:80...
* Connected to v4.ipv6test.app (64:ff9b::36c0:333c) port 80 (#0)
> GET / HTTP/1.1

##
## <Output truncated for brevity>
##
```

兩個 curl 指令都成功執行。請注意，由於 NAT64 和 DNS64 共同運作，因此透過 IPv6 連線至僅支援 IPv4 的網站是可行的。

檢查來源 IP

讓我們使用 IP 反射服務，檢查目的地觀察到的來源 IP。

```
<username>@gua-instance:~$ curl -6 v4.ipv6test.app  
  
<username>@gua-instance:~$ curl -6 v6.ipv6test.app
```

輸出範例

```
<username>@gua-instance:~$ curl -6 v4.ipv6test.app  
34.47.60.91  
  
<username>@gua-instance:~$ curl -6 v6.ipv6test.app  
2600:1900:40e0:6f:0:1::
```

回報的 IPv6 位址應與執行個體的 IPv6 位址相符。這個位址應與執行個體上「ip -6 address」指令的輸出內容相符。舉例來說

```
<username>@gua-instance:~$ ip -6 addr  
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 state UNKNOWN qlen 1000  
    inet6 ::1/128 scope host noprefixroute  
        valid_lft forever preferred_lft forever  
2: ens4: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 state UP qlen 1000  
    inet6 2600:1900:40e0:6f:0:1::/128 scope global dynamic noprefixroute  
        valid_lft 79912sec preferred_lft 79912sec  
    inet6 fe80::86:d9ff:fe34:27ed/64 scope link  
        valid_lft forever preferred_lft forever
```

附註：請先結束所有 SSH 工作階段，再繼續操作。

不過，由於 Cloud NAT 閘道會在傳輸至網際網路前執行 NAT64 功能，因此回報的 IPv4 位址應與 Cloud NAT 閘道的外部 IP 位址相符。您可以在 Cloud Shell 中執行下列 gcloud 指令，確認執行狀況：

```
gcloud compute routers get-nat-ip-info \  
    nat64-router \  
    --region=$regionname
```

輸出範例

```
result:  
- natIpInfoMappings:  
  - mode: AUTO  
    natIp: 34.47.60.91  
    usage: IN_USE  
    natName: nat64-natgw
```

請注意，輸出內容中回報的「natip」與 IP 反射網站收到的輸出內容相符。

從 ULA 執行個體測試 DNS64/NAT64

首先，透過 SSH 連線至 ULA 執行個體「ula-instance」

```
gcloud compute ssh ula-instance --project $projectname --zone $zonename  
  
<username>@ula-instance:~$
```

DNS 測試

測試僅支援 IPv6 的網站 DNS 解析情形 (例如 [v6.ipv6test.app](#)，但任何僅支援 IPv6 的網站都應產生類似結果)。

```
<username>@ula-instance:~$ host -t AAAA v6.ipv6test.app  
<username>@ula-instance:~$ host -t A v6.ipv6test.app
```

預期只會傳回 IPv6 AAAA 回覆。

輸出範例

```
<username>@ula-instance:~$ host -t AAAA v6.ipv6test.app  
v6.ipv6test.app has IPv6 address 2600:9000:269f:1000:9:ec55:a1c0:93a1  
v6.ipv6test.app has IPv6 address 2600:9000:269f:6600:9:ec55:a1c0:93a1  
v6.ipv6test.app has IPv6 address 2600:9000:269f:b600:9:ec55:a1c0:93a1  
v6.ipv6test.app has IPv6 address 2600:9000:269f:3e00:9:ec55:a1c0:93a1  
v6.ipv6test.app has IPv6 address 2600:9000:269f:9c00:9:ec55:a1c0:93a1  
v6.ipv6test.app has IPv6 address 2600:9000:269f:b200:9:ec55:a1c0:93a1  
v6.ipv6test.app has IPv6 address 2600:9000:269f:a600:9:ec55:a1c0:93a1  
v6.ipv6test.app has IPv6 address 2600:9000:269f:1400:9:ec55:a1c0:93a1  
  
<username>@ula-instance:~$ host -t A v6.ipv6test.app  
v6.ipv6test.app has no A record
```

測試僅支援 IPv4 的網站 (例如 [v4.ipv6test.app](#)) 的 DNS 解析。您預期會收到 A 記錄 (原始 IPv4) 和 DNS64 使用已知前置碼 64:ff9b::/96 合成的 AAAA 記錄。

```
<username>@ula-instance:~$ host -t AAAA v4.ipv6test.app  
<username>@ula-instance:~$ host -t A v4.ipv6test.app
```

輸出範例

```
<username>@gua-instance:~$ host -t AAAA v4.ipv6test.app
v4.ipv6test.app has IPv6 address 64:ff9b::36c0:3318
v4.ipv6test.app has IPv6 address 64:ff9b::36c0:3344
v4.ipv6test.app has IPv6 address 64:ff9b::36c0:333c
v4.ipv6test.app has IPv6 address 64:ff9b::36c0:3326

<username>@gua-instance:~$ host -t A v4.ipv6test.app
v4.ipv6test.app has address 54.192.51.68
v4.ipv6test.app has address 54.192.51.24
v4.ipv6test.app has address 54.192.51.60
v4.ipv6test.app has address 54.192.51.38
```

在上述範例中，十進位的 IPv4 位址 (54.192.51.38) 會轉換為十六進位的 (36 c0 33 26)，因此我們預期 AAAA 記錄的答案會是 (64:ff9b::36c0:3326)，這與我們收到的其中一個 AAAA 答案相符。

Curl 測試

讓我們使用 curl 測試與相同 v4 和 v6 端點的實際連線。

首先，讓我們證明無法透過 IPv4 存取，因為執行個體僅支援 IPv6。

```
<username>@ula-instance:~$ curl -vv -4 --connect-timeout 10 v6.ipv6test.app
<username>@ula-instance:~$ curl -vv -4 --connect-timeout 10 v4.ipv6test.app
```

但兩個 cURL 都會失敗。失敗原因各不相同。以下是輸出範例。

```
<username>@ula-instance:~$ curl -vv -4 v6.ipv6test.app
* Could not resolve host: v6.ipv6test.app
* Closing connection 0
curl: (6) Could not resolve host: v6.ipv6test.app

<username>@ula-instance:~$ curl -vv -4 --connect-timeout 10 v4.ipv6test.app
* Trying 54.192.51.68:80...
* ipv4 connect timeout after 4993ms, move on!
* Trying 54.192.51.38:80...
* ipv4 connect timeout after 2496ms, move on!
* Trying 54.192.51.24:80...
* ipv4 connect timeout after 1248ms, move on!
* Trying 54.192.51.60:80...
* Connection timeout after 10000 ms
* Closing connection 0
curl: (28) Connection timeout after 10000 ms
```

由於 A 記錄的 DNS 解析失敗 (如 DNS 測試所示)，因此對僅支援 IPv6 端點的 IPv4 curl 會失敗。IPv4 專用端點的 IPv4 cURL 會失敗，因為 IPv6 專用執行個體無法連線至任何 IPv4 位址，因此會發生逾時。

現在，讓我們測試 IPv6 的可連線性。

```
<username>@ula-instance:~$ curl -vv -6 v6.ipv6test.app
```

```
<username>@ula-instance:~$ curl -vv -6 v4.ipv6test.app
```

以下是輸出範例。

```
<username>@ula-instance:~$ curl -vv -6 v6.ipv6test.app
* Trying [2600:9000:20be:c000:9:ec55:alc0:93a1]:80...
* connect to 2600:9000:20be:c000:9:ec55:alc0:93a1 port 80 failed: Connection timed out
* Trying [2600:9000:20be:f000:9:ec55:alc0:93a1]:80...
* ipv6 connect timeout after 84507ms, move on!
* Trying [2600:9000:20be:ae00:9:ec55:alc0:93a1]:80...
* ipv6 connect timeout after 42253ms, move on!
* Trying [2600:9000:20be:2000:9:ec55:alc0:93a1]:80...
* ipv6 connect timeout after 21126ms, move on!
* Trying [2600:9000:20be:b600:9:ec55:alc0:93a1]:80...
* ipv6 connect timeout after 10563ms, move on!
* Trying [2600:9000:20be:7600:9:ec55:alc0:93a1]:80...
* ipv6 connect timeout after 5282ms, move on!
* Trying [2600:9000:20be:b000:9:ec55:alc0:93a1]:80...
* ipv6 connect timeout after 2640ms, move on!
* Trying [2600:9000:20be:3400:9:ec55:alc0:93a1]:80...
* ipv6 connect timeout after 2642ms, move on!
* Failed to connect to v6.ipv6test.app port 80 after 300361 ms: Timeout was reached
* Closing connection 0

<username>@ula-instance:~$ curl -vv -6 v4.ipv6test.app
* Trying [64:ff9b::36c0:333c]:80...
* Connected to v4.ipv6test.app (64:ff9b::36c0:333c) port 80 (#0)
> GET / HTTP/1.1

##
## <Output truncated for brevity>
##
```

由於 ULA 子網路無法直接連上網際網路，因此對僅支援 IPv6 的網站執行 curl 會失敗。由於 DNS64 和 NAT64 對於 GUA 和 ULA 執行個體的運作方式相同，因此對僅限 IPv4 的網站執行 curl 會成功；唯一的要求是執行個體僅限 IPv6。

附註：請先結束所有 SSH 工作階段，再繼續操作。

從雙重堆疊 ULA 執行個體測試 DNS64/NAT64

首先，請透過 SSH 連線至雙堆疊 ULA 執行個體「dualstack-ula-instance」。我們需要使用「-tunnel-through-iap」旗標，強制 gcloud 使用 IAP 的 IPv4 位址。

```
gcloud compute ssh dualstack-ula-instance --project $projectname --zone $zonename --tunnel-through-iap

<username>@dualstack-ula-instance:~$
```

現在使用「host」公用程式測試 DNS64。

```
<username>@dualstack-ula-instance:~$ host v4.ipv6test.app
v4.ipv6test.app has address 54.192.51.38
v4.ipv6test.app has address 54.192.51.24
v4.ipv6test.app has address 54.192.51.68
v4.ipv6test.app has address 54.192.51.60

<username>@dualstack-ula-instance:~$ host v6.ipv6test.app
v6.ipv6test.app has IPv6 address 2600:9000:269f:fc00:9:ec55:a1c0:93a1
v6.ipv6test.app has IPv6 address 2600:9000:269f:1c00:9:ec55:a1c0:93a1
v6.ipv6test.app has IPv6 address 2600:9000:269f:a200:9:ec55:a1c0:93a1
v6.ipv6test.app has IPv6 address 2600:9000:269f:8a00:9:ec55:a1c0:93a1
v6.ipv6test.app has IPv6 address 2600:9000:269f:c800:9:ec55:a1c0:93a1
v6.ipv6test.app has IPv6 address 2600:9000:269f:c200:9:ec55:a1c0:93a1
v6.ipv6test.app has IPv6 address 2600:9000:269f:5800:9:ec55:a1c0:93a1
v6.ipv6test.app has IPv6 address 2600:9000:269f:dc00:9:ec55:a1c0:93a1
```

請注意，僅限 IPv4 的網站現在只會傳回 IPv4 位址，不會再傳回合成 DNS64 答案。這是因為 DNS64 只會套用至僅支援 IPv6 的執行個體，不會評估雙重堆疊執行個體。

如要略過 DNS64，請在 /etc/hosts 檔案中新增項目，測試 NAT64 是否正常運作。在雙堆疊執行個體中執行下列指令：

```
<username>@dualstack-ula-instance:~$ echo '64:ff9b::36c0:3326 v4.ipv6test.app' | sudo tee -a /etc/hosts
```

接著使用 curl 測試透過 IPv6 連線至 IPv4 網站

```
<username>@dualstack-ula-instance:~$ curl -vv -6 --connect-timeout 10 v4.ipv6test.app
```

上述指令的輸出範例如下：

```
<username>@dualstack-ula-instance:~$ curl -vv -6 --connect-timeout 10 v4.ipv6test.app

* Trying [64:ff9b::36c0:3326]:80...
* ipv6 connect timeout after 10000ms, move on!
* Failed to connect to v4.ipv6test.app port 80 after 10001 ms: Timeout was reached
* Closing connection 0
curl: (28) Failed to connect to v4.ipv6test.app port 80 after 10001 ms: Timeout was reached
```

curl 應會逾時，因為與 DNS64 相同，NAT64 也需要執行個體僅支援 IPv6 才能套用。

注意：請先結束 SSH 工作階段，再繼續操作。

如要確認 NAT64 實際上並未套用至雙堆疊執行個體，請使用「get-nat-mapping」指令列出 NAT 閘道套用的所有連接埠對應。在 Cloud Shell 中執行下列指令：

```
gcloud compute routers get-nat-mapping-info \
  nat64-router --region $regionname \
  --project $projectname
```

輸出內容應類似於下列程式碼片段：

```
---
instanceName: gua-instance
interfaceNatMappings:
- natIpPortRanges:
  - 34.47.60.91:1024-1055
  numTotalDrainNatPorts: 0
  numTotalNatPorts: 32
  sourceAliasIpRange: ''
  sourceVirtualIp: '2600:1900:40e0:6f:0:1::'
- natIpPortRanges:
  - 34.47.60.91:32768-32799
  numTotalDrainNatPorts: 0
  numTotalNatPorts: 32
  sourceAliasIpRange: ''
  sourceVirtualIp: '2600:1900:40e0:6f:0:1::'
---
instanceName: ula-instance
interfaceNatMappings:
- natIpPortRanges:
  - 34.47.60.91:1056-1087
  numTotalDrainNatPorts: 0
  numTotalNatPorts: 32
  sourceAliasIpRange: ''
  sourceVirtualIp: fd20:9c2:93fc:2800:0:0:0:0
- natIpPortRanges:
  - 34.47.60.91:32800-32831
  numTotalDrainNatPorts: 0
  numTotalNatPorts: 32
  sourceAliasIpRange: ''
  sourceVirtualIp: fd20:9c2:93fc:2800:0:0:0:0
```

NAT 輸出內容顯示 NAT64 閘道只為僅限 IPv6 的 GUA 和 ULA 執行個體分配通訊埠，但未為雙重堆疊執行個體分配通訊埠。

步驟 8 · 約 0 分鐘

8. 清理

清理 Cloud Router

在 Cloud Shell 中執行下列操作：

```
gcloud compute routers delete nat64-router \
  --region $regionname \
  --project $projectname --quiet
```

取消繫結並清理 DNS 政策

在 Cloud Shell 中執行下列操作：

```
gcloud beta dns policies update allow-dns64 \
  --networks="" \
  --project $projectname
gcloud beta dns policies delete allow-dns64 \
  --project $projectname --quiet
```

清理執行個體

在 Cloud Shell 中執行下列操作 (注意，我們使用 gcloud beta 啟用 no-graceful-shutdown 標記，以便更快刪除執行個體)：

```
gcloud beta compute instances delete gua-instance \
  --zone $zonename \
  --no-graceful-shutdown \
  --project=$projectname --quiet

gcloud beta compute instances delete ula-instance \
  --zone $zonename \
  --no-graceful-shutdown \
  --project=$projectname --quiet

gcloud beta compute instances delete dualstack-ula-instance \
  --zone $zonename \
  --no-graceful-shutdown \
  --project=$projectname --quiet
```

清理子網路

在 Cloud Shell 中執行下列操作：


```
gcloud compute networks subnets delete gua-v6only-subnet \
  --project=$projectname --quiet \
  --region=$regionname

gcloud compute networks subnets delete ula-v6only-subnet \
  --project=$projectname --quiet \
  --region=$regionname

gcloud compute networks subnets delete ula-dualstack-subnet \
  --project=$projectname --quiet \
  --region=$regionname
```

清理防火牆規則

在 Cloud Shell 中執行下列操作：

```
gcloud compute firewall-rules delete allow-v6-iap \
  --project=$projectname \
  --quiet

gcloud compute firewall-rules delete allow-v6-ssh-ula \
  --project=$projectname \
  --quiet

gcloud compute firewall-rules delete allow-v4-iap \
  --project=$projectname \
  --quiet
```

清理虛擬私有雲

在 Cloud Shell 中執行下列操作：

```
gcloud compute networks delete ipv6-only-vpc \
  --project=$projectname \
  --quiet
```

清理 IAM 權限和服務帳戶

在 Cloud Shell 中執行下列操作：

```
gcloud projects remove-iam-policy-binding $projectname \
  --member=serviceAccount:ipv6-codelab@$projectname.iam.gserviceaccount.com \
  --role=roles/compute.instanceAdmin.v1

gcloud iam service-accounts delete \
  ipv6-codelab@$projectname.iam.gserviceaccount.com \
  --quiet \
  --project $projectname
```


步驟 9 · 約 0 分鐘

9. 恭喜

您已成功使用 NAT64 和 DNS64，讓僅支援 IPv6 的執行個體連線至網際網路上僅支援 IPv4 的目的地。

後續步驟

查看一些程式碼研究室...

- [使用 IPv6 位址從地端部署主機存取 Google API](#)
- [IP 定址選項：IPv4 和 IPv6](#)
- [使用 IPv6 靜態路由的下一個躍點執行個體、下一個躍點位址和下一個躍點閘道](#)

延伸閱讀與影片

- [Google Cloud 上的 IPv6 簡介](#)

參考文件

- [支援靜態路徑的 IPv6](#)
 - [IPv6 子網路](#)
 - [NAT64/DNS64 總覽](#)
-